

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks:50

Annexure A

Coding challenge

Code of Conduct:

I D.Uday Kiran(192425161) certify that ~~this submission is my original work and that I have~~ adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

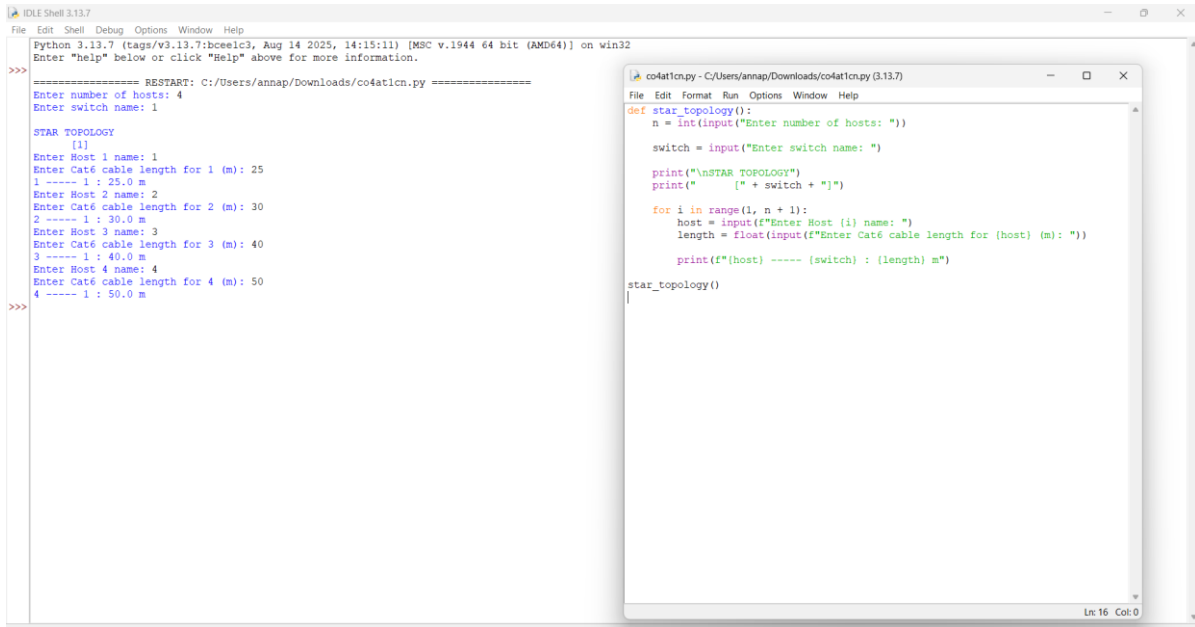
Signature of the student:

D.Uday Kiran

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



The screenshot shows two windows in the Python IDLE environment. The left window is the Shell, showing the execution of a script named `co4at1cn.py`. The user has entered the number of hosts as 4 and the switch name as 1. The script has generated a star topology diagram with the following output:

```
===== RESTART: C:/Users/annap/Downloads/co4at1cn.py =====
Enter number of hosts: 4
Enter switch name: 1

STAR TOPOLOGY
[1]
Enter Host 1 name: 1
Enter Cat6 cable length for 1 (m): 25
1 ----- 1 : 25.0 m
Enter Host 2 name: 2
Enter Cat6 cable length for 2 (m): 30
2 ----- 1 : 30.0 m
Enter Host 3 name: 3
Enter Cat6 cable length for 3 (m): 40
3 ----- 1 : 40.0 m
Enter Host 4 name: 4
Enter Cat6 cable length for 4 (m): 50
4 ----- 1 : 50.0 m
```

The right window is the Editor, showing the source code of `co4at1cn.py`:

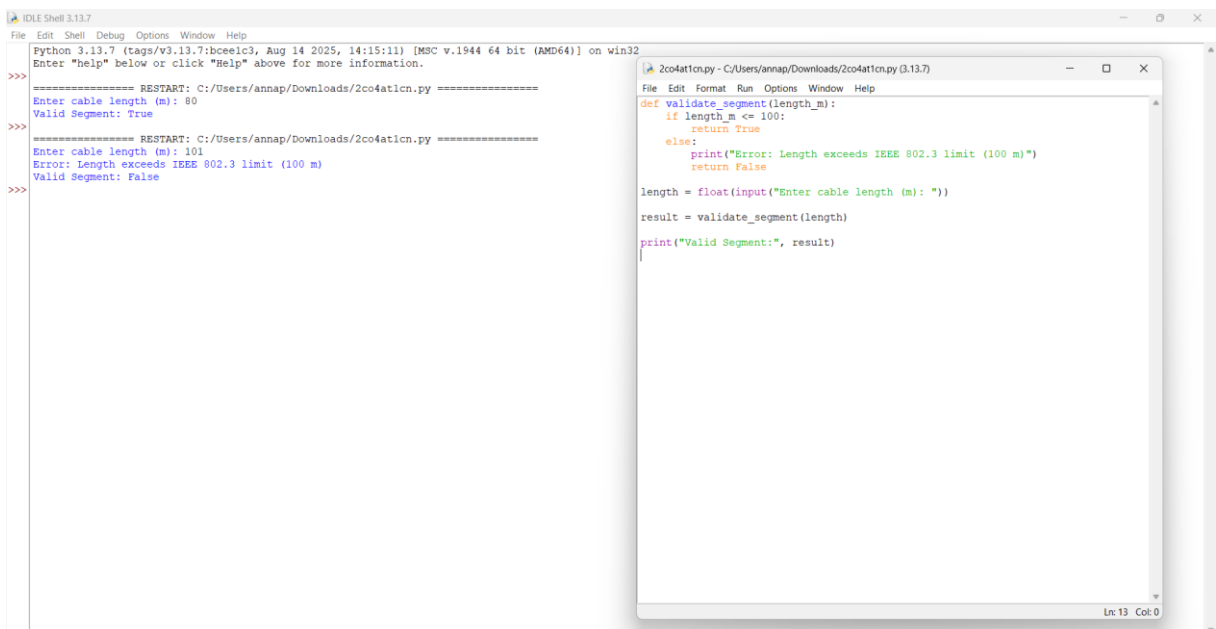
```
def star_topology():
    n = int(input("Enter number of hosts: "))
    switch = input("Enter switch name: ")

    print("\nSTAR TOPOLOGY")
    print("[" + switch + "]")

    for i in range(1, n + 1):
        host = input(f"Enter Host (i) name: ")
        length = float(input(f"Enter Cat6 cable length for (host) (m): "))
        print(f"({host}) ----- ({switch}) : ({length}) m")

star_topology()
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.



The screenshot shows two windows in the Python IDLE environment. The left window is the Shell, showing the execution of a script named `2co4at1cn.py`. The user has entered cable lengths of 80, 101, and 100 meters. The script has generated the following output:

```
===== RESTART: C:/Users/annap/Downloads/2co4at1cn.py =====
Enter cable length (m): 80
Valid Segment: True
===== RESTART: C:/Users/annap/Downloads/2co4at1cn.py =====
Enter cable length (m): 101
Error: Length exceeds IEEE 802.3 limit (100 m)
Valid Segment: False
===== RESTART: C:/Users/annap/Downloads/2co4at1cn.py =====
Enter cable length (m): 100
Valid Segment: True
```

The right window is the Editor, showing the source code of `2co4at1cn.py`:

```
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print("Error: Length exceeds IEEE 802.3 limit (100 m)")
        return False

length = float(input("Enter cable length (m): "))
result = validate_segment(length)
print("Valid Segment:", result)
```

- Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.

```

Python 3.13.7 (tags/v3.13.7:bce61c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/annap/Downloads/3co4at1cn.py =====
Enter number of hosts: 4
Enter Host 1 name: H1
Enter Host 2 name: H2
Enter Host 3 name: H3
Enter Host 4 name: H4

Collision between H3 and H2
H3 Backoff: 5 ms
H2 Backoff: 4 ms

Collision between H4 and H2
H4 Backoff: 2 ms
H2 Backoff: 2 ms

Collision between H4 and H1
H4 Backoff: 7 ms
H1 Backoff: 3 ms

Collision between H2 and H3
H2 Backoff: 6 ms
H3 Backoff: 6 ms

Collision between H4 and H3
H4 Backoff: 8 ms
H3 Backoff: 5 ms
Log saved in csma_log.txt
>>>

```

- Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

```

Python 3.13.7 (tags/v3.13.7:bce61c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/annap/Downloads/4co4at1cn.py =====
Enter message from Host1: hi this is robo
Enter message from Host2: hi brroooo

STAR SWITCH
Switch MAC Address: AA:BB:CC:DD:EE:01
Waiting for hosts...

Host1 sending: Host1 Host2 sending: hi this is roboconnectedhi brroooo

MAC Address Table: {'Host1': '00:11:22:33:44:01'}

Host2 connected
MAC Address Table: {'Host1': '00:11:22:33:44:01', 'Host2': '00:11:22:33:44:02'}

Frame received from Host1
Message: hi this is robo
Host2 received: MAC Address Table: Forwarded by switch: hi this is robo('Host1': '00:11:22:33:44:01', 'Host2': '00:11:22:33:44:02')

Frame received from Host2
Message: hi brroooo
Host1 received: MAC Address Table: Forwarded by switch: hi brroooo('Host1': '00:11:22:33:44:01', 'Host2': '00:11:22:33:44:02')
>>>

```

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

The screenshot shows a Python IDE with two windows. The main window displays the execution of a script, and a smaller window shows the source code of the script.

Main Window Output:

```
Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/annap/Downloads/Sco4aticn.py =====
Enter number of hosts: 5
Enter Host 1 name: H1
Enter Host 2 name: H2
Enter Host 3 name: H3
Enter Host 4 name: H4
Enter Host 5 name: H5
Enter source host: H1
Enter message: HI THIS IS ROBO!

Broadcasting...
Message sent from H1 to H2: HI THIS IS ROBO!
Message sent from H1 to H3: HI THIS IS ROBO!
Message sent from H1 to H4: HI THIS IS ROBO!
Message sent from H1 to H5: HI THIS IS ROBO!
>>>
```

Source Code Window (Sco4aticn.py):

```
class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)

    def remove_port(self, host):
        if host in self.ports:
            self.ports.remove(host)

    def broadcast(self, source, message):
        print("\nBroadcasting...")
        for host in self.ports:
            if host != source:
                print(f"Message sent from {source} to {host}: {message}")

switch = StarSwitch()

n = int(input("Enter number of hosts: "))

for i in range(n):
    host = input(f"Enter Host {i+1} name: ")
    switch.add_port(host)

source = input("Enter source host: ")
msg = input("Enter message: ")

switch.broadcast(source, msg)
```